MY WEEK 3 NOTES

AI Engineering Bootcamp

Agents · The Agent Loop · Multi-Agent · MCP · A2A

📌 **What's in this notebook**

1) The Agent Loop by hand (raw SDK) 2) What agents really are 3) Single → Multi-agent design 4) ADK (the build tool) 5) MCP (tools for agents) 6) A2A (agents talking to agents) 7) Full system + evaluation 8) Bonus: the 5 layers of AI engineering 9) Homework assignment (step-by-step)

① The Big Idea — What is an Agent, really?

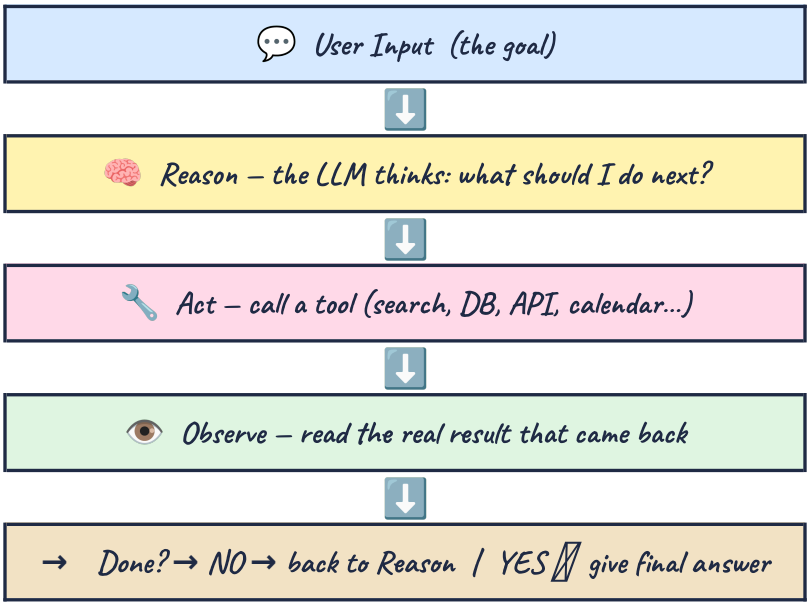
A **chatbot** just does **input → output**. One question, one answer, done.

An **agent** is different — it does *THINK → ACT → OBSERVE → REPEAT*, in a loop, until the job is actually done.

✏️ Think of it like a person doing a task, not a search engine

If you ask a person to 'find out if flight AI202 is delayed', they don't just answer from memory. They open a tab, check the site, read the result, and THEN tell you the answer — and if the site is down, they try another source. That back-and-forth is exactly the agent loop.

✏️ The Agent Loop (a.k.a. ReAct: Reason + Act)



② The Four Steps, Named Properly

The lesson breaks the loop into 4 concrete steps — memorize WHO does each one:

Step	Who does it	What actually happens
Think	The Model	Given the goal + history + tool schemas, the model decides: answer directly, OR call a named tool with arguments.
Act	YOUR CODE	Your runtime actually runs the tool (HTTP call, DB query, calendar API, search). The model NEVER runs code itself — you do.
Observe	The Result	The tool's output (or its error!) gets appended back into the conversation as a 'tool result' message that the model can read.
Decide again	The Loop	Model sees the REAL result and either calls another tool, or returns the

Step	Who does it	What actually happens
		final answer. This responsiveness is what makes it an agent, not a script.

✏️ Slides call this the '4 Core Components' of an agent

Same idea, viewed as building blocks instead of steps:

Component	Icon	What it means
Instructions	📝	The system prompt — what the agent should do, its constraints, its personality.
Model	🧠	The reasoning engine — GPT, Gemini, Claude, Llama. Chooses what to do next.
Tools	🔧	Plain functions the agent can call: search, query a DB, send an email, hit an API.
Memory	📈	Short-term (this session) and long-term (across sessions) — what the agent remembers.

> ✅ **Use an agent when...**
>
> the task is multi-step, the request is ambiguous, it needs to touch external systems, or the path isn't known ahead of time.

> ❌ **Skip the agent when...**
>
> it's a simple prompt→response, the workflow is fully deterministic, or you need very low latency. A fixed script will be cheaper, faster, and more predictable.

③ Why Build the Loop 'By Hand' First?

Every framework you'll ever use — **LangGraph**, **Google ADK**, **Claude tool use**, **OpenAI Assistants** — is just *ergonomics* wrapped around this exact same heartbeat (Think→Act→Observe→Decide).

If you only ever meet agents inside a framework, debugging feels like pure magic when something breaks. But if you've written the raw while-loop yourself even once, you can read ANY framework's execution trace and immediately spot exactly where Think, Act, or Observe went wrong.

> 🎯 **Golden rule**
>
> Don't skip the raw loop and jump straight to a framework. Understand the engine before you drive the automatic car.

✏️ What you actually build with the raw SDK

Using the plain OpenAI or Anthropic SDK (no LangGraph, no ADK yet):

1. Register 1-2 tools, each with a **JSON schema** describing its name, purpose, and arguments.
2. Run a while-loop that: sends messages to the model → checks if it returned tool_calls → executes them in your code → appends results back to the conversation.
3. Stop the loop when the model returns plain text with NO tool calls (it's done), OR when you hit a max-iteration guard (safety net).

> 🚨 *CRITICAL — Always bound the loop!*
>
> Unbounded agents can burn tokens and money forever if they get stuck calling tools in circles. Cap iterations (e.g. 8-12), log every single tool call, and FAIL CLOSED (stop and report) when the cap is hit.

✏️ Common mistakes to avoid (straight from the lesson)

- Skipping the raw loop and jumping straight into a framework.
- Letting the agent loop run with NO max-iteration or cost guard.
- Treating tool errors as silent failures — instead they should be observations the model itself gets to see and react to.
- Calling something an 'agent' when it's really just a fixed 3-step pipeline (no real decision-making).

④ Workflow vs Agent — The Line That Matters

⚙️ *WORKFLOW*	🤖 *AGENT*
Follows steps YOU defined in advance. Use when the path is known.	*Decides its OWN steps toward a goal. Use when judgement/branching depends on tool results you can't script ahead.*

> 💡 *The professional skill*
>
> Agents cost more (tokens, latency) and fail differently than workflows. Choosing correctly between the two — for each part of a system — IS the actual skill you're being trained on this week.

⑤ From Single Agent to Multi-Agent

One agent with 15+ tools eventually breaks — not because of the tool COUNT itself, but because the model's context window fills up with junk and it loses focus. The fix: split by domain. Each agent gets a focused job, short instructions, and a limited toolset.

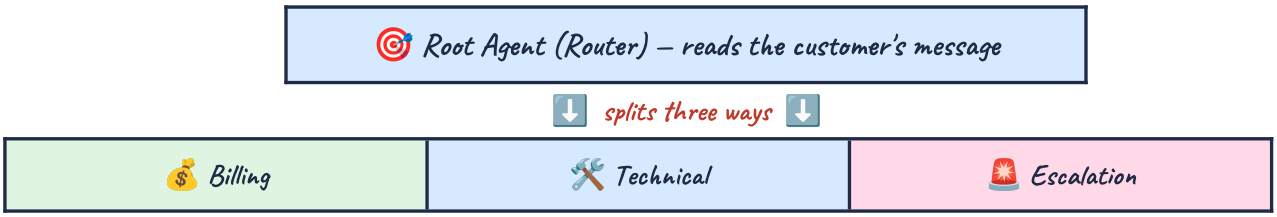
✏️ Four Multi-Agent Patterns

Pattern	How it works	Good for
🎯 Router / Delegation	Root agent reads intent, delegates to the right specialist.	Customer support (we build this one!)
➡️ Sequential Pipeline	Fixed order: A → B → C, every time.	Content generation, data processing
🔀 Parallel Fan-Out	Multiple agents run at once, results get merged.	Research, aggregation
🔄 Loop / Refinement	Produce → review → loop again until quality is met.	Writing, code generation

✏️ Key design decisions you must make

- How many agents? → **Split by domain, not by task.**
- Who routes? → An LLM (adaptive, flexible) or a fixed workflow (deterministic, predictable).
- Data sharing? → Session state, output keys, or an external database.
- Failure handling? → Escalation, fallback to a human, or retry.

✏️ The example we build all session: a support system



Why this works: Modularity, Specialization, Reusability, Testability, Scalability — each specialist can be built, tested, and improved on its own, without touching the others.

⑥ Introducing ADK — Your Build Tool

ADK = **Agent Development Kit** — Google's open-source framework for building and deploying AI agents. It's designed to feel like *software development*, not 'prompt engineering with extra steps.'

Feature	What it means
🔌 Open Source	Python, TypeScript, Go, Java — multi-language from day one.
🤖 Model-Agnostic	Use Gemini, Claude, Ollama, LiteLLM, or basically any model.
☁️ Deployment-Agnostic	Run it locally, on Cloud Run, GKE, or Vertex AI.
🔗 Native MCP + A2A	Built-in support for BOTH open protocols that matter in production.

✏️ Why ADK over LangGraph / CrewAI / AutoGen?

Framework	Strength	Trade-off
LangGraph	Graph-based orchestration, strong community	Steeper learning curve
CrewAI	Role-based multi-agent, easy to start	Less control over execution flow
AutoGen	Research-grade multi-agent conversations	Complex setup, Microsoft-centric
ADK	Multi-language, native MCP + A2A, built-in eval + dev UI	Newer ecosystem, still maturing

✏️ Your first agent — it's just a Python function!

The LLM reads the function's NAME and DOCSTRING to decide when to call it. No manual JSON schema needed — ADK builds that for you automatically.

```
from google.adk.agents import Agent

def lookup_customer(email: str) -> dict:
    "Look up customer account information by email."
    return {"name": "Jane Doe", "plan": "Pro"}

def check_order_status(order_id: str) -> dict:
    "Check the current status of an order."
    return {"order_id": order_id, "status": "shipped"}

support_agent = Agent(
    name="support_agent",
    model="gemini-2.5-flash",
    instruction="You are a helpful customer support agent.",
    tools=[lookup_customer, check_order_status],
)
```

✏️ Your first MULTI-agent system in ADK

The root agent reads the `description` of each sub-agent and decides who should handle the query — no explicit if/else routing logic required.

```
billing_agent = Agent(
    name="billing_agent", model="gemini-2.5-flash",
    description="Handles billing: invoices, payments, refunds.",
    instruction="Help customers with billing issues.",
    tools=[lookup_invoice, process_refund],
)

technical_agent = Agent(
    name="technical_agent", model="gemini-2.5-flash",
    description="Handles technical issues: bugs, outages, how-to.",
    instruction="Help customers with technical problems.",
    tools=[search_knowledge_base, check_system_status],
)
```

```
root_agent = Agent(
    name="support_router", model="gemini-2.5-flash",
    instruction="Route customer queries to the right specialist.",
    sub_agents=[billing_agent, technical_agent],
)
```

> 👀 *Demo 1 — things to notice*
>
> The router agent has NO tools — it ONLY routes. Each specialist has a focused instruction + a limited toolset. The ADK Dev UI's 'trace view' shows you the full execution path so you can literally watch the routing happen.

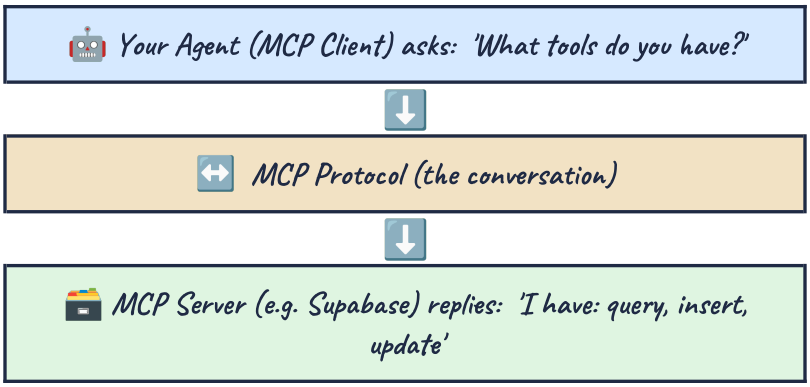
⑦ MCP — Giving Agents Tools

So far we hardcoded tools as plain Python functions. In the real world, agents need to reach databases, SaaS platforms, and internal APIs. MCP is the open standard for that.

> 🔌 *Think of MCP as 'USB for AI Agents'*
>
> One universal connector, instead of writing a custom cable (integration code) for every single device (tool/database/API).

✏️ Client-Server architecture

- **MCP Server** — exposes tools, resources, and data (e.g. Supabase, Asana, a filesystem).
- **MCP Client** — your agent. It discovers and uses whatever tools the server offers.



Direction	What it means
📥 Consuming	Your ADK agent connects to an MCP server, auto-discovers its tools, and calls them transparently via McpToolset.
📤 Exposing	You wrap YOUR ADK tools as an MCP server. Then any MCP client can use them — Claude Desktop, Cursor, or another agent.

✏️ What can you connect via MCP?

Category	Examples
🗃️ Databases	Supabase, Spanner, AlloyDB, Postgres (via MCP Toolbox)
📋 Project Mgmt	Asana (30+ tools), Atlassian (Jira + Confluence)
☁️ Google Cloud	BigQuery, Bigtable, Cloud API Registry
🎨 Media	Imagen, Veo, Chirp 3 HD, Lyria (Genmedia MCP)
📁 File Systems	Local filesystem access, document reading
🔗 Custom APIs	Any API via Apigee, or build your own with FastMCP

✏️ MCP in ADK — no hardcoded database functions!

```
from google.adk.agents import Agent
from google.adk.tools.mcp_tool import McpToolset, StdioConnectionParams
from mcp.client.stdio import StdioServerParameters

supabase_mcp = McpToolset(
    connection_params=StdioConnectionParams(
        server_params=StdioServerParameters(
            command="npx",
            args=["-y", "@supabase/mcp-server-supabase@latest",
                  "--access-token", TOKEN],
        ),
    ),
)

billing_agent = Agent(
    model="gemini-2.5-flash", name="billing_agent_mcp",
    instruction="You are a billing specialist with real database access.",
    tools=[supabase_mcp],
)
```

🚨 Security — always filter tools in production

Don't hand an agent write access if it only ever needs to read! Use tool_filter to whitelist exactly which tools it may call.

```
McpToolset(
    connection_params=StdioConnectionParams(...),
    tool_filter=["read_file", "list_directory", "search_files"],
    # Only these tools are available to the agent
)
```

⑧ A2A — Agents Talking to Agents

MCP connects agents to TOOLS. But what connects agents to OTHER AGENTS, possibly across completely different systems? That's A2A — an open standard.

Pattern	Relationship	Analogy
🔧 MCP	agent ↔️ tool	Giving an agent a toolkit
👥 Multi-Agent	agent ↔️ agent, same app	Teammates in the same room
🌐 A2A	agent ↔️ agent, across network	Calling a colleague at another office

A2A lets a Python agent talk to a Java agent, or your support system call a partner company's shipping agent — without either side needing to know how the other one was built.

✏️ How A2A works — Expose, then Consume

STEP 1 — EXPOSE: Your Agent → to_a2a() → becomes an A2A Server (gets an Agent Card + a network endpoint)

⬇️

STEP 2 — CONSUME: Your Agent → RemoteA2aAgent(url) → uses the Remote Agent (feels just like a local sub-agent — ADK handles the networking)

✏️ Agent Cards — how agents discover each other

Every A2A agent publishes an Agent Card: a JSON file describing what it does, what inputs it accepts, what outputs it returns, and where to reach it (its URL).

> 🪪 *Think of it as...*
>
> An API spec, but for agents. ADK auto-generates the Agent Card for you the moment you call to_a2a().

✏️ A2A in code

Expose (make network-accessible):

```
from google.adk.agents import Agent
from google.adk.a2a.utils.agent_to_a2a import to_a2a

shipping_agent = Agent(
    name="shipping_status_agent", model="gemini-2.5-flash",
    instruction="Look up shipping status.",
    tools=[get_shipping_status],
)
```

```
app = to_a2a(shipping_agent, port=8001)
# uvicorn agent:app --port 8001
```

Consume (use someone else's remote agent):

```
from google.adk.agents.remote_a2a_agent import RemoteA2aAgent

remote_shipping = RemoteA2aAgent(
    name="shipping_agent",
    agent_card="http://localhost:8001",
)

root_agent = Agent(
    name="customer_support",
    sub_agents=[billing, technical, remote_shipping],
)
```

✏️ Real-world A2A use cases

- **Microservices:** Order Agent ↔️ Inventory Agent ↔️ Shipping Agent ↔️ Payment Agent, each its own service.
- **Cross-org:** your support agent calls a partner's warranty-verification agent — you don't know their tech stack, and you don't need to.
- **Cross-language:** a Python orchestrator talks to a Java compliance agent and a Go data-processing agent.
- **Third-party services:** a financial data provider exposes live stock prices via an A2A agent; your advisor agent just consumes it.

⑨ Full System — Putting It All Together

✏️ Decision guide — which pattern do I need?

You need to...	Use
Connect an agent to a database, API, or file system	MCP
Split a complex task into focused agents in one app	Multi-Agent

You need to...	Use
Call an agent running as a separate service	A2A
Build a predictable pipeline (step 1 → 2 → 3)	Workflow Agents
Let the LLM decide which agent handles a request	LLM Delegation
Make your agent usable by anyone, any framework	A2A (expose)

✏️ Test scenarios (from Demo 4)

- "What's the status of my order #1234?" → **Billing → MCP → Supabase**
- "My app keeps crashing on login" → **Technical → MCP → Knowledge Base**
- "Where is my package?" → **A2A → Remote Shipping Agent**

✏️ Evaluation & Deployment — before/after you ship

Check	Question it answers
🎯 Response Quality	Is the final answer correct? (defined via test cases with expected outputs)
👁️ Trajectory	Did the agent call the right tools AND route to the correct sub-agent?

Deployment target	Best for
ADK Dev UI	Local dev & debugging
CLI	Quick testing, CI/CD
Agent Engine	Managed production (Vertex AI)
Cloud Run	Containerized, custom infra
GKE	Kubernetes, multi-service

All deployment targets support A2A — agents can be exposed and consumed no matter where they actually run.

⑩ Bonus: 'New Kids on the Block' — Terms Coming Soon

Awareness only — not required for homework, but good to recognise in the wild.

✏️ The Five Layers of AI Engineering

Today's session sat mostly at the Loop and Graph layers. The stack is cumulative — you don't 'graduate' away from lower layers, you build on top of them.

Layer	What you engineer	Your role	Core question
Prompt	The single request	Operator	Am I asking well?
Context	What the model sees	Editor	Does it have the right info?
Harness	Tools, memory, scaffolding	Toolmaker	Can it act and remember?
Loop	The cycle one agent repeats	System designer	When does it check its work & stop?
Graph	Coordination between many agents	Org designer	Who does what, in what order, sharing what state?

⚠️ *Key warning*

If your nodes (agents) are weak on their own, wiring them into a fancy org chart just gives you a weak org. A 15-tool agent doesn't break because of the tool COUNT — it breaks because its context fills with junk.

✏️ Graph Engineering — nodes, edges, state

- **Nodes** — units that do one job each: a specialised agent, or a plain deterministic step.
- **Edges** — the routing between nodes: straight, conditional, fan-out, fan-in.
- **State** — the object travelling along the edges that every node reads from and writes to.

Researcher {task}	*Writer {task, notes}*	*Reviewer {task, notes, draft}*

A pass ships; a reject loops back to the Writer. Mapping to ADK: SequentialAgent = a straight edge, ParallelAgent = fan-out then fan-in, LoopAgent = an edge back to itself, and an LlmAgent with sub_agents = a conditional edge decided at runtime.

🔑 *Rule of thumb*

A loop is a single-node graph with an edge back to itself. Most real work is one job with one verifier — master the loop first, and only split into a full graph when the work forces your hand.

✏️ Loop Engineering (a third meaning of 'loop')

Not the ReAct cycle, not ADK's LoopAgent — this is about designing the SYSTEM that prompts the agent, instead of being the person who prompts it. Boris Cherny (head of Claude Code, Anthropic) says he doesn't prompt Claude any more — he has loops running that prompt Claude and figure out what to do. His job is to write loops.

Building block	What it gives the loop
1. Automations	Scheduled discovery and triage — the heartbeat.
2. Worktrees	So parallel agents don't collide on the same files.
3. Skills	Project knowledge written down instead of guessed every session.
4. Connectors	= MCP, the thing from Section 4.

Building block	What it gives the loop
5. Sub-agents	So the one who writes isn't the one who checks.
6. State	A file/board outside the conversation — the model forgets between runs.

⚠️ *Real cost*

Token cost blows up fast, and verification still stays with YOU. A loop running unattended is a loop making mistakes unattended. 'Done' is a claim, not a proof.

⭐ Three Things to Remember

🔧 *MCP agent ↔️ tool How agents access tools & data*	👥 *Multi-Agent agent ↔️ agent, same app How agents work together locally*	🌐 *A2A agent ↔️ agent, across network How agents collaborate across boundaries*

These three patterns compose. Start with the concepts, pick the right pattern, then implement with ADK.

✍️ 📚 HOMEWORK ASSIGNMENT

Build a Multi-Agent Customer Support System with MCP and A2A

> 🎯 **Goal in one sentence**
>
> One router agent + at least 2 specialists, where at least one specialist talks to a real database via MCP, and a returns agent runs as a separate service you reach via A2A.

✏️ STEP 1 — Set up Supabase

4. Create a new Supabase project (free tier is fine).
5. Create three tables: **customers**, **orders**, **support_tickets**.
6. Seed each table with 10+ realistic sample records (names, order IDs, statuses, ticket descriptions) so your agent has something real to query.

✏️ STEP 2 — Build the Multi-Agent System in ADK

7. Install ADK and set up your project skeleton (`pip install google-adk`, or the language of your choice).
8. Write a ROOT (router) agent with NO tools of its own — its only job is to read intent and delegate.
9. Write at least 2 specialist sub-agents (e.g. billing_agent, technical_agent), each with a short instruction and a small, focused toolset.
10. Connect AT LEAST ONE specialist to Supabase using McpToolset, so it can run real queries instead of hardcoded Python functions.
11. Test it locally with the ADK Dev UI and watch the trace view to confirm routing actually works.

✏️ STEP 3 — Build a Returns Agent and expose it via A2A

12. Create a separate agent (a separate file/service) with two tools: check_return_eligibility and initiate_return.
13. Expose it over the network using to_a2a(returns_agent, port=8002) (or a port of your choice).
14. Run it as its own process — e.g. `uvicorn agent:app --port 8002` — confirm you can see its Agent Card at that URL.

✏️ STEP 4 — Connect the Returns Agent and test everything

15. In your main support system, add a RemoteA2aAgent pointing at the Returns Agent's URL, and include it in root_agent's sub_agents list alongside your local specialists.
16. Test THREE scenarios end-to-end:
    - **Billing (MCP)** — e.g. "What's the status of my order #1234?"
    - **Returns (A2A)** — e.g. "I want to return my order, is it eligible?"
    - **Escalation** — e.g. a message that no specialist can handle, and check it escalates sensibly.
17. Record the ADK Dev UI trace view for each scenario so you can SEE the routing + tool calls happen.

✏️ Deliverables checklist

- ✅ Working code pushed to a GitHub repo.
- ✅ A README with an architecture diagram (you can literally redraw the box diagrams from these notes).
- ✅ A 2–3 minute screen recording showing all 3 test scenarios running inside the ADK Dev UI.

✏️ Stretch goals (optional, for extra credit / practice)

- Add a LoopAgent for a draft/review refinement cycle.
- Apply tool_filter to make your MCP connections read-only where appropriate.
- Write 5+ formal ADK eval test cases (response quality + trajectory).
- Expose your OWN root agent via A2A, so a classmate's agent could call yours.

💪 Before you submit — sanity check yourself

Can you explain, in your own words, WHY you chose MCP for one connection and A2A for another? If yes, you've actually understood the lesson, not just copy-pasted the code.

— end of notes — you've got this! 🚀